‘’’

LP-III Assignment 4 (Group A - DAA)

Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

Name: Tanmay Sandip Bothara

Roll No: 23 Batch: BE_Comp_A1

’’’

```
def knapsack_01(n, values, weights, W):
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    dp[i - 1][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    selected_items = []
    i, w = n, W

    while i > 0 and w > 0:
        if dp[i][w] != dp[i - 1][w]:
            selected_items.append(i - 1)
            w -= weights[i - 1]
        i -= 1

    selected_items.reverse()
```

```
    return dp[n][W], selected_items



if __name__ == "__main__":
    n = int(input("Enter number of items: "))

    values = []
    weights = []

    for i in range(n):
        value = int(input(f"Enter value of item {i + 1}: "))
        weight = int(input(f"Enter weight of item {i + 1}: "))

        values.append(value)
        weights.append(weight)

    W = int(input("Enter knapsack capacity: "))

    max_value, selected_items = knapsack_01(n, values, weights, W)

    print("Maximum value:", max_value)
    print("Selected items (0-based index):", selected_items)
    print("Selected item numbers:", [i + 1 for i in selected_items])
```

‘’’

OUTPUT:-

Enter number of items: 3

Enter value of item 1: 60

Enter weight of item 1: 10

Enter value of item 2: 100

Enter weight of item 2: 20

Enter value of item 3: 120

Enter weight of item 3: 30

Enter knapsack capacity: 50

Maximum value: 220

Selected items (0-based index): [1, 2]

Selected item numbers: [2, 3]

‘’’